# Binomial Heaps

An Efficient Data Structure for Merge-Heavy Systems

Muttakin Ahmed Md. Sium Md. Fuad Al Alam

April 9, 2026

CSE200, Bangladesh University of Engineering and Technology

# Contents

**1 Motivation & Basics** **2**
1.1 The Mailing Service Problem . . . 2
1.2 The Easiest Solution vs. Real-World Constraints . . . 2
1.3 Implementing a Priority Queue: Binary Heaps . . . 2
1.4 The Problem of Scale and the Merging Bottleneck . . . 2
1.5 The Mathematical Analogy & Solution . . . 3

**2 Binomial Trees & Merging** **4**
2.1 The Building Blocks: Binomial Trees . . . 4
2.1.1 Standard Properties of a Binomial Forest . . . 4
2.2 Representing Arbitrary Numbers of Nodes . . . 4
2.3 Merging and Carries . . . 4

**3 Operations & Complexity** **6**
3.1 The Power of Merge . . . 6
3.2 Standard Operations . . . 6
3.2.1 Find Minimum . . . 6
3.2.2 Add (Insert) . . . 6
3.2.3 Decrease Key . . . 6
3.2.4 Extract Minimum . . . 6
3.3 Complexity Summary and Comparisons . . . 7

# Chapter 1

# Motivation & Basics

*Presented by Muttakin Ahmed*

## 1.1 The Mailing Service Problem

Imagine we are architecting a mailing service. In this system, we receive mail (requests) from users and must process them sequentially. A foundational constraint is that our system can only handle one request at a time. When simultaneous requests flood the server, a critical question arises: how do we decide which one gets served first?

## 1.2 The Easiest Solution vs. Real-World Constraints

The easiest approach is implementing a First-In, First-Out (FIFO) methodology utilizing a standard queue. However, this fails us in practical application because not all users are equal. A simple queue cannot prioritize urgent traffic over regular traffic, which means it cannot properly accommodate premium subscribers.

We need a data structure that always serves the highest priority item first, regardless of arrival time. This structure is called a Priority Queue, which generally supports operations such as Insert(Request) and Extract-Max() or Extract-Min().

## 1.3 Implementing a Priority Queue: Binary Heaps

A standard implementation for a priority queue is a Binary Heap, which is a perfect or almost complete binary tree. It must satisfy the Heap Property:

- **Max-Heap:** Every parent is strictly greater than or equal to its children.
- **Min-Heap:** Every parent is strictly less than or equal to its children.

Using a Binary Heap is highly efficient for basic tasks. Adding (inserting) an element takes $\mathcal{O}(\log n)$ time, and removing (extracting) takes $\mathcal{O}(\log n)$ time as well.

## 1.4 The Problem of Scale and the Merging Bottleneck

As our mailing service expands and becomes wildly popular, we may upgrade the servers to use multithreading. Each thread handles a chunk of traffic and builds its own separate heap. When threads complete their chunk, their separate heaps must be merged into the unified master heap.

This introduces the primary problem with Binary Heaps: merging two binary heaps is extremely slow. Adding elements one by one takes $\mathcal{O}(n \log n)$ time, while creating a brand new array and rebuilding from scratch takes linear $\mathcal{O}(n)$ time.

## 1.5 The Mathematical Analogy & Solution

Imagine physically merging a pile of 25 sticks with a pile of 18 sticks. If added one-by-one, it takes exactly 18 operations, running in strict linear time $\mathcal{O}(n)$.

However, if we group items in powers of two—mirroring binary numbers—we can optimize this. For example, $25_{10} = 11001_2$ and $18_{10} = 10010_2$. By adding these bit-by-bit mathematically, it only takes approximately 5 steps. Each "bit" represents a different magnitude (1, 2, 4, 8...), dropping the time complexity to logarithmic $\mathcal{O}(\log n)$.

This concept is the foundation of the **Binomial Heap**. Instead of one monolithic tree, we use a forest (collection) of structured trees representing powers of 2, allowing us to literally merge heaps by "adding bits".

# Chapter 2

# Binomial Trees & Merging

*Presented by Md. Sium*

## 2.1 The Building Blocks: Binomial Trees

A Binomial Heap is composed of special trees called Binomial Trees, denoted as $B_k$. Every tree has exactly $2^k$ nodes. We build larger trees by snapping two smaller, identical trees together.

- A $B_0$ tree has $2^0 = 1$ node.
- A $B_1$ tree is made by making one $B_0$ tree the child of another's root, resulting in $2^1 = 2$ nodes.
- A $B_k$ tree is always created by joining a $B_{k-1}$ tree with another $B_{k-1}$ tree.

### 2.1.1 Standard Properties of a Binomial Forest

Standard properties dictate that a $B_k$ tree has a depth of exactly $k$. Furthermore, the root has exactly $k$ children. These children are roots of binomial trees of orders $k-1, k-2, \ldots, 0$, which are ordered from left to right.

## 2.2 Representing Arbitrary Numbers of Nodes

If Binomial Trees must have exactly $2^k$ nodes, how do we store an odd number of requests? We leverage binary representation. Suppose our heap has exactly 13 nodes. The number 13 in binary is $1101_2$, which breaks down into $8 + 4 + 1$. Therefore, our forest must be made up of exactly: $B_3$, $B_2$, and $B_0$ trees.

Consider merging a heap containing exactly 7 nodes with one containing exactly 19 nodes:

- 19 in binary is $10011_2$, breaking down to $16 + 2 + 1$. The forest consists of $B_4$, $B_1$, and $B_0$.
- 7 in binary is $0111_2$, breaking down to $4 + 2 + 1$. The forest consists of $B_2$, $B_1$, and $B_0$.

## 2.3 Merging and Carries

When merging two trees of the same order, we handle it much like encountering a $1 + 1$ carry operation in binary arithmetic:

- Keep the smaller root to maintain the min-heap property.

- Make the larger root its leftmost child.
- The order of the tree increases by 1 (carrying the bit to the left).
- We repeat this step until there are no remaining duplicate tree orders.

Because we only traverse up to $\log n$ trees, a full merge operation takes at most $\mathcal{O}(\log n)$ steps, which is a massive upgrade over $\mathcal{O}(n)$.

# Chapter 3

# Operations & Complexity

*Presented by Md. Fuad Al Alam*

## 3.1 The Power of Merge

The golden rule of Binomial Heaps is that the entire data structure revolves around the $\mathcal{O}(\log n)$ *Merge* operation. For nearly every major operation—aside from finding a node—we just configure the data slightly and then forcefully call *Merge* to handle the heavy lifting.

## 3.2 Standard Operations

### 3.2.1 Find Minimum

The smallest value in any $B_k$ tree is strictly its root node. Therefore, the smallest value in the entire forest must be one of the root nodes. Scanning the linked list of up to $\log n$ trees takes $\mathcal{O}(\log n)$ time. We usually optimize this by keeping an external pointer directly targeting the global minimum root. By updating this pointer during merges and extractions, Find Minimum becomes a true $\mathcal{O}(1)$ operation.

### 3.2.2 Add (Insert)

Adding a singular request to our priority queue is straightforward. We treat the new element not as a standalone item, but as a brand new Binomial Heap containing exactly one node ($B_0$). Because *Merge* runs in $\mathcal{O}(\log n)$ time, our Add operation naturally runs in bounded $\mathcal{O}(\log n)$ time as well.

### 3.2.3 Decrease Key

When a premium subscriber upgrades their tier, their priority queue index decreases. We adjust the node's value, but this might violate the min-heap property. To fix this, we push the decreased value upward toward the root, swapping with its parent until the heap property is restored, an action known as "Bubbling up". Because the absolute maximum depth of a tree containing $n$ elements is $\log n$, we can bubble up at most $\log n$ times, yielding a time complexity of $\mathcal{O}(\log n)$.

### 3.2.4 Extract Minimum

To process and remove the highest priority mail, we extract the global minimum by removing the minimum root from the linked list of roots. If you delete the root of a $B_k$ tree, its remaining children are exactly the trees $B_{k-1}, B_{k-2}, \ldots B_0$. The orphaned children form a perfectly valid,

standalone Binomial forest. We treat the children as a second heap and call Merge() to fuse it back into the main forest in $\mathcal{O}(\log n)$ time.

## 3.3 Complexity Summary and Comparisons

For multithreaded or distributed systems where heaps must coalesce, Binomial Heaps reign supreme over standard Binary Heaps. In a standard Binary Heap, the *Merge* operation costs $\mathcal{O}(n)$, whereas it only costs $\mathcal{O}(\log n)$ in a Binomial Heap.

Could we push the limits even further? Yes, using Fibonacci Heaps. Fibonacci Heaps employ severe "lazy evaluation". They throw trees together in $\mathcal{O}(1)$ time and delay actually merging them until an *Extract* call absolutely forces it. This achieves an unheard of $\mathcal{O}(1)$ amortized time for Insert, Merge, and Decrease Key.

However, in practice, Fibonacci Heaps suffer from immense constant overhead factors. They rely heavily on pointers, which causes massive CPU cache misses, and their implementation is exceptionally complex and prone to errors. Ultimately, Binomial Heaps strike the perfect, practical balance between optimal merge performance and sensible engineering complexity.